

บทที่ 4

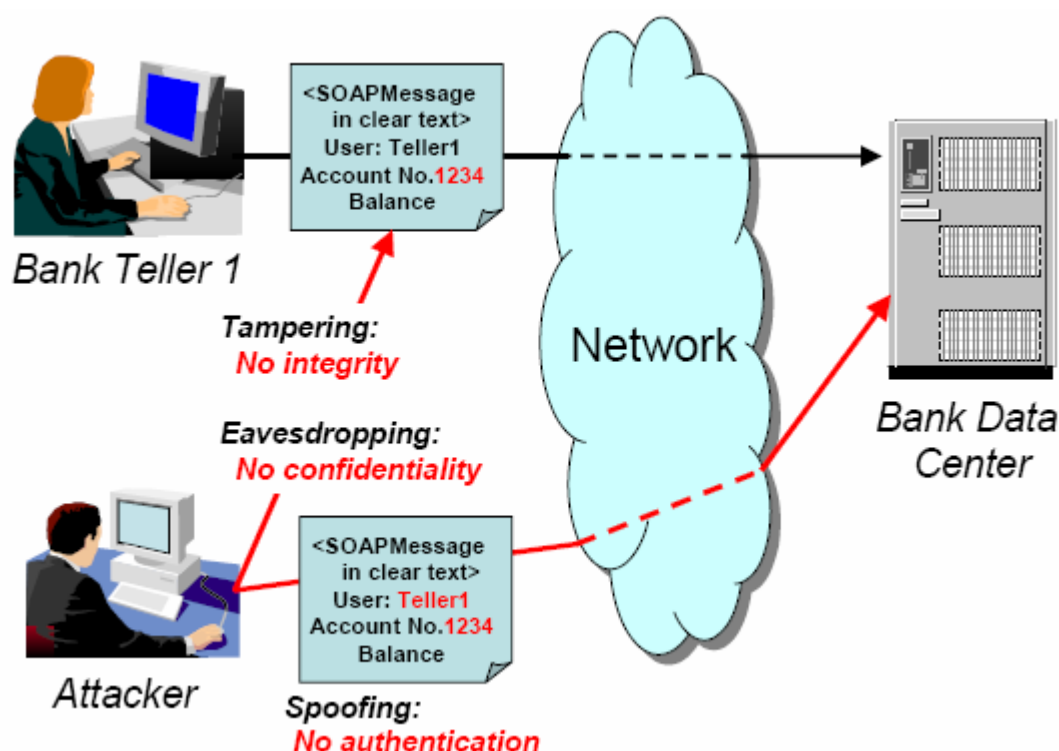
WS-Security

4.1 นิยามของความมั่นคงปลอดภัยในคอมพิวเตอร์

ในปัจจุบันระบบคอมพิวเตอร์ได้ถูกคุกคามมากขึ้นทั้งจากไวรัสคอมพิวเตอร์หรือจากผู้ไม่ดี ซึ่งความมั่นคงปลอดภัยคอมพิวเตอร์ (Computer Security) ช่วยปกป้องเครื่องคอมพิวเตอร์ รวมไปถึงถึงอุปกรณ์ต่างๆที่เกี่ยวข้อง และที่สำคัญยังสามารถช่วยปกป้องข้อมูลที่ได้จัดเก็บไว้ในระบบ หรือใช้ในความปลอดภัยทางข้อมูลสารสนเทศ (Information Security) ก็ได้

จุดประสงค์หลักของความปลอดภัยทางข้อมูลคือ

- **การรักษาความลับ (Confidentiality)** คือการรับรองว่าจะมีการเก็บข้อมูลไว้เป็นความลับ และผู้มีสิทธิเท่านั้นจึงจะเข้าถึงข้อมูลนั้นได้
- **การรักษาความสมบูรณ์ (Integrity)** คือการรับรองว่าข้อมูลจะไม่ถูกเปลี่ยนแปลงหรือทำลายไม่ว่าจะเป็นโดย อุบัติเหตุหรือ โดยเจตนา
- **ความพร้อมใช้ (Availability)** คือการรับรองว่าข้อมูลและบริการการสื่อสารต่าง ๆ พร้อมที่จะใช้ได้ในเวลาที่ต้องการใช้งาน
- **การห้ามปฏิเสธความรับผิดชอบ (Non-Repudiation)** คือวิธีการสื่อสารซึ่งผู้ส่งข้อมูลได้รับหลักฐานว่าได้มีการส่งข้อมูลแล้วและผู้รับก็ได้รับการยืนยันว่าผู้ส่งเป็นใคร ดังนั้นทั้งผู้ส่งและผู้รับจะไม่สามารถปฏิเสธได้ว่าไม่มีความเกี่ยวข้องกับข้อมูลดังกล่าวในภายหลัง
- **การพิสูจน์ตัวตน (Authentication)** คือขั้นตอนการยืนยันความถูกต้องของหลักฐาน (identity) ที่แสดงว่าเป็นบุคคลที่กล่าวอ้างจริง
- **การกำหนดสิทธิ์ (Authorization)** คือขั้นตอนในการอนุญาตให้แต่ละบุคคลสามารถเข้าถึงข้อมูลหรือระบบใดได้บ้าง ก่อนอื่นต้องทราบก่อนว่าบุคคลที่กล่าวอ้างนั้นคือใครตามขั้นตอนการพิสูจน์ตัวตนและต้องให้แน่ใจด้วยการพิสูจน์ตัวตนนั้นถูกต้อง
- **การตรวจสอบ (Audit)** คือการตรวจสอบหลักฐานทางอิเล็กทรอนิกส์ ซึ่งสามารถใช้ในการติดตามการดำเนินการเพื่อตรวจสอบความถูกต้องและแม่นยำ ตัวอย่างเช่นการตรวจสอบบัญชีชื่อผู้ใช้ โดยผู้ตรวจบัญชี ซึ่งการตรวจสอบความถูกต้องของการดำเนินการเพื่อให้แน่ใจว่าหลักฐานทางอิเล็กทรอนิกส์นั้นได้ถูกสร้างและสั่งให้ทำงานโดยบุคคลที่ได้รับอนุญาต และในการเชื่อมต่อเหตุการณ์เข้ากับบุคคลจะต้องทำการตรวจสอบหลักฐานของบุคคลนั้นด้วย ซึ่งถือเป็นหลักการพื้นฐานของขั้นตอนการทำงานของการทำงานของการพิสูจน์ตัวตนด้วย



รูปที่ 4.1 การส่ง SOAP Message ที่ไม่มีการรักษาความปลอดภัย

จากภาพเป็นการจำลองเหตุการณ์ที่พนักงานแบงก์ (bank teller) ซึ่งเปรียบได้กับ web service client จะส่งข้อมูลไปยังแบงก์สำนักงานใหญ่ (bank data center) ซึ่งเปรียบได้กับ web service provider โดยข้อมูลจะอยู่ในรูปของ SOAP message ที่ไม่มีการรักษาความปลอดภัยใดๆ จะเห็นได้ว่าจากสถานการณ์นี้จะเกิดความเสียหายหลักได้ 3 ประการ คือ

- **Spoofing : No authentication** ในกรณีนี้ attacker สามารถที่เปลี่ยนแปลงแก้ไข SOAP message และส่ง message ดังกล่าวไปยัง web service provider ได้ เสมือนตัวเองเป็นพนักงานแบงก์ (bank teller) โดย attacker อาจจะแอบอ้างว่าตนเองเป็นพนักงานแบงก์ (bank teller) แล้วทำการส่ง message เช่น message สำหรับการถอนเงินจากบัญชีของลูกค้ารายอื่นไปยังแบงก์สำนักงานใหญ่ (bank data center) ได้

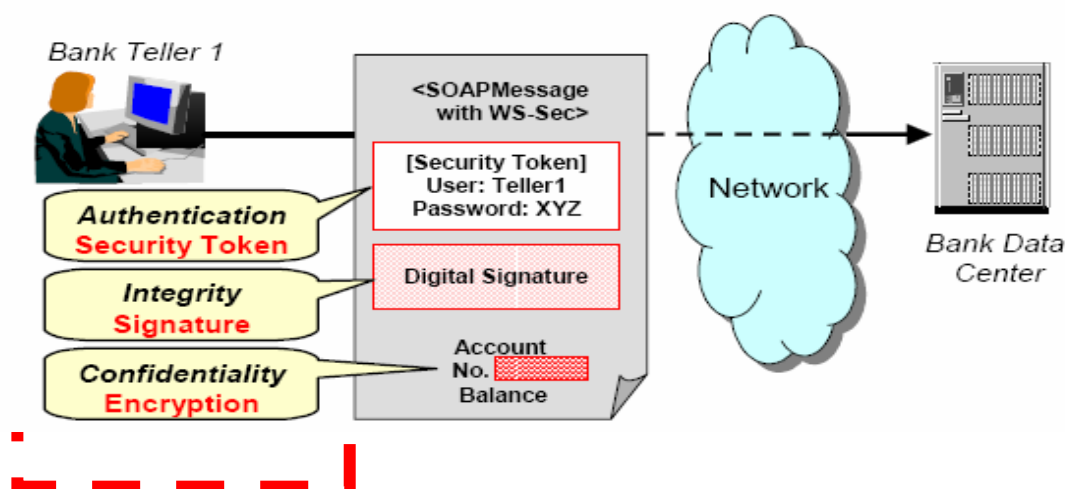
วิธีการแก้ไข สามารถใช้การพิสูจน์ตัวตน (Authentication) ในการแก้ปัญหาดังกล่าวเพื่อใช้ในการตรวจว่า message ที่ส่งมานั้นมาจากบุคคลนั้นจริงๆหรือไม่

- **Eavesdropping : No Confidentiality** ในกรณีนี้ attacker สามารถดักจับ SOAP message และอ่านข้อมูลที่อยู่ใน message นั้นได้ เพราะ message ไม่ได้ทำการเข้ารหัสเอาไว้ ดังนั้นข้อมูลต่างๆที่สำคัญของลูกค้าหรือข้อมูลของแบงก์อาจจะไปอยู่ในมือของบุคคลอื่นได้ เช่น หมายเลขของบัญชี

วิธีการแก้ไข สามารถใช้การรักษาความลับ (Confidentiality) ในการแก้ปัญหาดังกล่าว เพื่อใช้ในการเข้ารหัสข้อมูลที่ส่งไป

- Tampering : No Integrity ในกรณีนี้ attacker สามารถดักจับ SOAP message และทำการเปลี่ยนแปลงข้อมูลใน message เช่น การฝากเงินไปยังบัญชีอื่นโดยการเปลี่ยนแปลง หมายเลขบัญชี ซึ่งที่ web service server เองก็ไม่มีตรวจสอบว่า message ที่ส่งมาถูกเปลี่ยนแปลงไปหรือไม่

วิธีการแก้ไข สามารถใช้การรักษาความสมบูรณ์ (Integrity) ในการแก้ปัญหาดังกล่าว เพื่อทำการตรวจสอบ message ที่ส่งมาว่าถูกทำการเปลี่ยนแปลงหรือไม่



รูปที่ 4.2 การส่ง SOAP Message ที่มีการรักษาความปลอดภัย

WS-Security ออกแบบมาเพื่อเพิ่มความปลอดภัยของข้อมูลในระดับ Message ในการส่งข้อมูลระหว่าง เว็บเซอร์วิส แบบ End-to-End นั่นหมายความว่า message จะมีความปลอดภัยถึงแม้จะมีการส่งผ่านหลายๆเซิร์ฟวิส (ผ่าน intermediaries) แสดงดังรูป และเพิ่มประสิทธิภาพในการส่ง SOAP Message ให้มีความน่าเชื่อถือ มีความถูกต้อง และ มีความปลอดภัย (integrity, confidentiality, authentication) ทำให้ข้อมูลมีความปลอดภัยตลอดเวลาไม่ว่าจะถูกส่งไปให้ Service ไหนก็ตาม

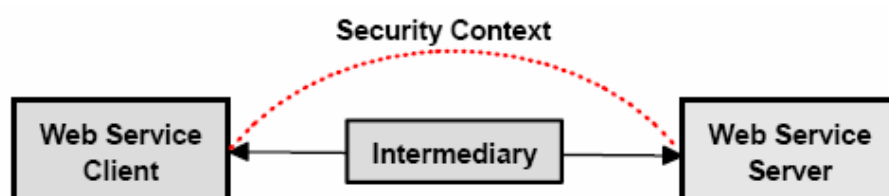
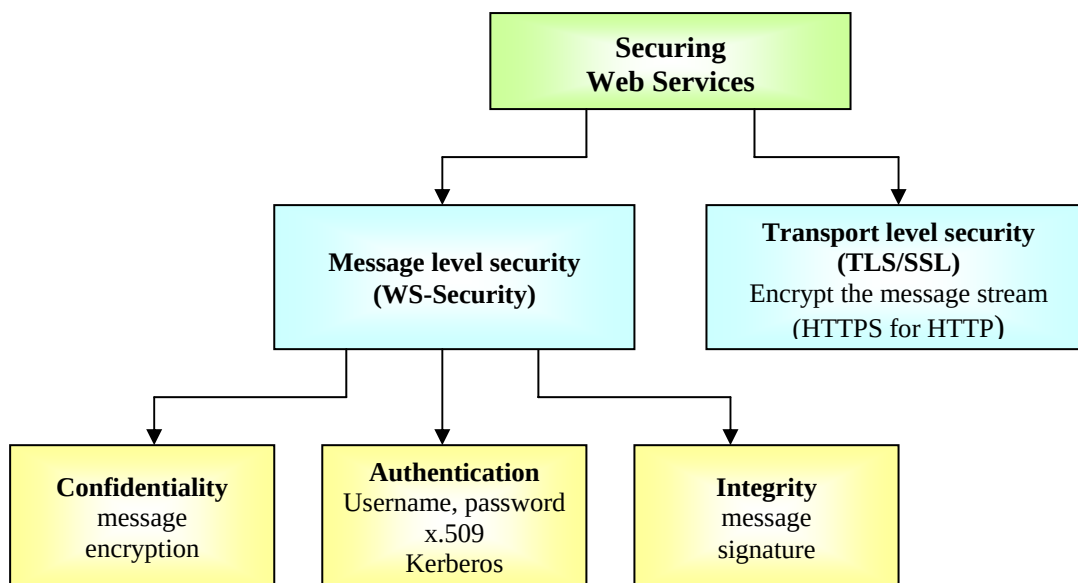


Figure 8-3 End-to-end security with message-level security

รูปที่ 4.3 การรักษาความปลอดภัยข้อมูลแบบ End-to-End โดยใช้ WS-security



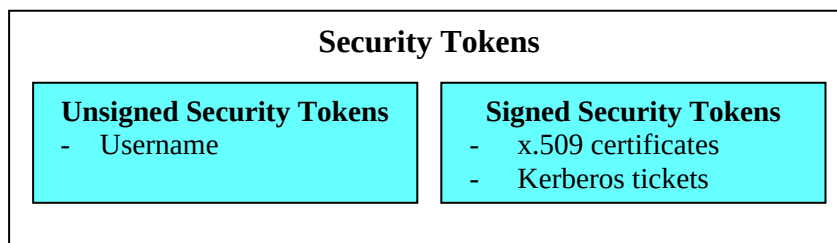
รูปที่ 4.4 โมเดล Securing Web Services

4.2 การพิสูจน์ตัวตน (Authentication)

WS-Security จะใช้ token ในการพิสูจน์ตัวตน เช่น การใช้ username token โดยจะมีการส่ง username และ password ไปพร้อมกับ SOAP Message เมื่อเว็บเซิร์ฟเวอร์ได้รับ username, password จะทำการตรวจสอบ ถ้าข้อมูลที่ได้ถูกต้อง message จะถูกนำไปประมวลผลต่อไป

Security Token ถูกเป็นออกเป็น 2 ประเภท คือ

1. Unsigned Security Token เป็น Token ที่ไม่มีผู้รับรอง เป็นเพียงการแสดงสิทธิ์ของตนเอง ไม่มีการรับรองแต่อย่างใด เช่น Username Token
2. Signed Security Token (Binary Security Token) เป็น Token ที่ได้รับการรับรองจากผู้ที่มีความน่าเชื่อถือ เช่น Kerberos Ticket หรือ x.509 Certificate



รูปที่ 4.5 ประเภทของ Token

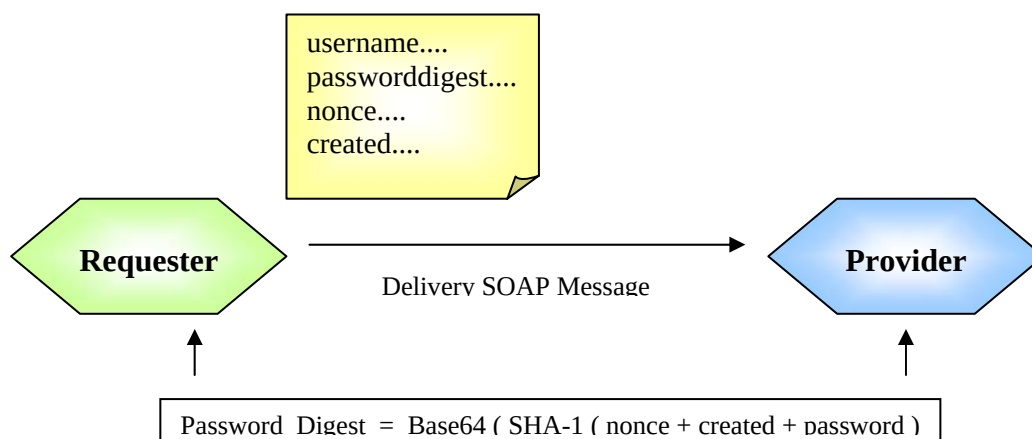
4.2.1 หลักการทำงานของ Username Token

Username Token สามารถส่งได้ทั้งแบบระบุรหัสผ่านโดยตรง (clear text) และแบบ digest โดยใช้ nonce โดยการส่งในแบบแรก SOAP Message ที่ส่งไปจะไม่ปลอดภัย เพราะแฮกเกอร์สามารถเข้าดักจับ (trap) password ได้ ยกเว้นว่า SOAP Message นั้นจะถูกส่งอยู่ในช่องทางที่ปลอดภัยอยู่แล้วเช่น SSL สำหรับวิธีแบบ digest โดยใช้ nonce เป็นวิธีการป้องกันการโจมตีแบบสแนก โดยผู้ส่งเป็นผู้สร้าง nonce ซึ่งเป็นค่าแบบสุ่มขึ้นมา โดยวิธีการนี้ทำให้ต้องมีเซิร์ฟเวอร์ที่มีแคช (cache) เพียงพอต่อการเก็บค่า nonce ที่ถูกใช้แล้วเอาไว้ทำให้สิ้นเปลืองรีซอร์ส (resource) จึงมีการใช้ควบคู่ไปกับ timestamp (created) วิธีนี้มีข้อดีคือ สามารถกำหนดลิ้มิตแคชของ nonce ให้อยู่ในช่วงเวลาที่ใหม่ (freshness) เท่านั้น ถ้ามีการใช้ nonce เกิดขึ้นจะมีการเพิ่มค่าของพาสเวิร์ดไดเจสต์ (Password_Digest value) เข้าไปใน SOAP Message ด้วย โดยค่าพาสเวิร์ดไดเจสต์หาได้โดย นำค่า nonce, created และ password มาทำการ concatenate แล้วทำการแฮชด้วยอัลกอริทึม SHA-1 หลังจากนั้นทำการ encode ด้วย BASE64 ดังสมการนี้

$$Password_Digest = Base64 (SHA-1 (nonce + created + password))$$

ขั้นตอนการส่ง SOAP Message ที่มีการใช้ nonce และ created

1. Requester สร้าง nonce ขึ้นมาพร้อมทั้งระบุเวลา timestamp (created) แล้วนำค่า password, nonce และ created มา concatenate แล้วทำการแฮชด้วย อัลกอริทึม SHA-1 หลังจากนั้นทำการ encode ด้วย BASE64 เพื่อหาค่าของ PasswordDigest แล้วนำค่าที่ได้ใส่ลงใน SOAP Message ที่จะส่งด้วย
2. เมื่อฝั่ง Provider ได้รับ SOAP Message จะตรวจสอบความถูกต้องของค่า PasswordDigest ที่ส่งมา โดยนำค่า password ที่เก็บไว้มา concatenate กับค่า nonce และ created ที่ส่งมากับ SOAP Message แล้วทำการแฮชด้วยอัลกอริทึม SHA-1 หลังจากนั้นทำการ encode ด้วย BASE64 ถ้าค่าที่ได้ตรงกันกับค่า PasswordDigest ที่ส่งมา message จะถูกนำไปประมวลผลต่อไป



รูปที่ 4.6 การส่ง SOAP Message ที่มีการใช้ nonce และ created

เมื่อมีการส่ง username token จากระบบหนึ่งไปอีกระบบหนึ่งโดยไม่มีการรักษาความปลอดภัยในการขนส่งข้อมูลเช่น HTTP อาจทำให้ token ถูกดักจับได้ทำให้เป็นจุดอ่อนในการโจมตีระบบได้ เว้นแต่จะใช้ XML digital signature หรือ XML Encryption เข้ามาช่วย

4.2.2 อีลีเมนต์ที่ใช้ในการส่งซีเคียวริตี้โทเคนแบบระบุรหัสผ่านโดยใช้ nonce

- <wsse:UsernameToken> แสดงถึงการ ใช้ Username Token
- <wsse:Username> แสดงถึงยูสเซอร์เนมที่ใช้
- <wsse:Password> แสดงถึงรหัสผ่านของยูสเซอร์
 - แอทริบิวต์ PasswordType ระบุว่าใช้รหัสผ่านเป็นแบบใดเจสต์
- <wsse:Nonce> เป็นค่าที่ไว้ป้องกันการโจมตีเมสเสจ (replay attack)
- <wsu:Created> เป็นค่าที่บอกว่าเมสเสจนี้ถูกสร้างขึ้นเมื่อไร (timestamp)

```

<S:Envelope xmlns:S="http://www.w3.org/2001/12/soap-envelope"
  xmlns:wsse="http://schemas.xmlsoap.org/ws/2003/06/secext">
  <S:Header>
    ...
    <wsse:Security>
      <wsse:UsernameToken
        xmlns:wsse="http://schemas.xmlsoap.org/ws/2003/06/secext"
        xmlns:wsu="http://schemas.xmlsoap.org/ws/2003/06/utility">
  
```

```

    <wsse:Username>NNK</wsse:Username>

    <wsse:Password Type="wsse:PasswordDigest">
        weYI3nXd8LjMNVksCKFV8t3rgHh3Rw==
    </wsse:Password>

    <wsse:Nonce>WScqanjCEAC4mQoBE07sAQ==</wsse:Nonce>

    <wsu:Created>2003-07-16T01:24:32Z</wsu:Created>

    </wsse:UsernameToken>

</wsse:Security>

...

</S:Header>

...

</S:Envelope>

```

รูปที่ 4.7 ตัวอย่างการใช้ Username Token แบบใช้ nonce

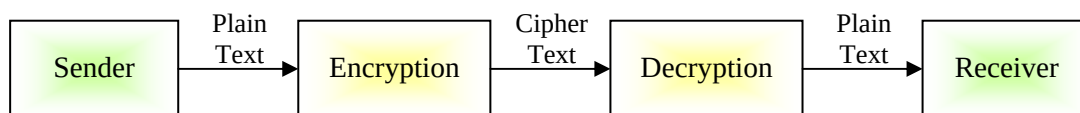
4.3 การรักษาความลับของข้อมูล (Confidentiality)

การรักษาความลับของข้อมูล คือ การรับรองว่าจะมีการเก็บข้อมูลไว้เป็นความลับ และผู้มีสิทธิเท่านั้นจึงจะเข้าถึงข้อมูลนั้นได้ WS-Security สามารถจัดการเกี่ยวกับการเข้ารหัสข้อมูลบางส่วน หรือทั้งหมดของ SOAP message ก่อนที่จะทำการส่งต่อไป ซึ่งมาตรฐานหนึ่งที่ถูกนำมาใช้คือ XML Encryption ซึ่งมาจาก W3C

4.3.1 หลักการทำงานของ XML Encryption

XML Encryption คล้ายกับการเข้ารหัสแบบต่างๆ ไป แต่ต่างกันตรงที่ว่า XML Encryption สามารถเลือกทำการเข้ารหัสเอกสารทั้งหมด หรือเลือกเฉพาะบางส่วนของข้อมูลในส่วนของเฮดเดอร์, บอดี หรือข้อมูลที่แนบมาของ SOAP message นั้น

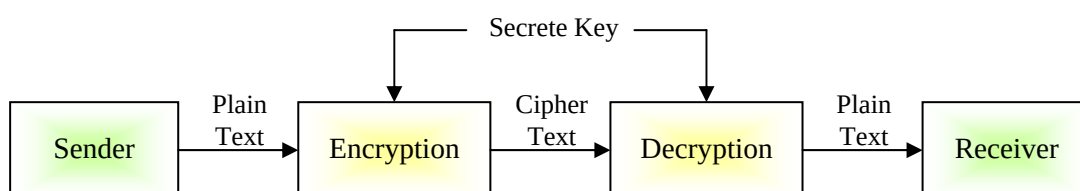
การเข้ารหัส คือ กระบวนการในการเปลี่ยนข้อมูลต้นฉบับให้อยู่ในอีกรูปแบบหนึ่งที่ไม่สามารถเข้าใจได้โดยง่าย ทำให้ผลลัพธ์ที่ได้จากการเข้ารหัสจะสามารถนำกลับไปเป็นข้อมูลต้นฉบับได้นั้นต้องใช้การถอดรหัส (Decryption) โดยเราจะเรียกข้อมูลต้นฉบับที่จะทำการเข้ารหัสว่า เคลียร์เท็กซ์ (Clear text) หรือ เฟลนเท็กซ์ (Plain text) และเรียกข้อมูลที่ทำการเข้ารหัสเรียบร้อยแล้วว่า ไซเฟอร์เท็กซ์ (Cipher text)



รูปที่ 4.8 การเข้ารหัสและถอดรหัส

การเข้ารหัสนั้นวิธีการหนึ่งที่ได้คือการเข้ารหัสและถอดรหัสโดยใช้กุญแจ มี 2 แบบ คือ การเข้ารหัสแบบสมมาตร (Symmetric Key Cryptography)

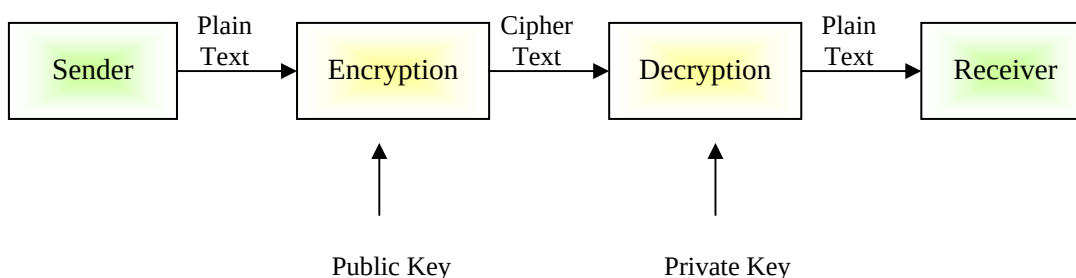
อัลกอริทึมแบบนี้จะใช้กุญแจที่เรียกว่า กุญแจลับ (Secret key) ซึ่งมีเพียงหนึ่งเดียวเพื่อใช้ในการเข้ารหัสและถอดรหัสข้อความที่ส่งไป



รูปที่ 4.9 การเข้ารหัสและถอดรหัสแบบสมมาตร

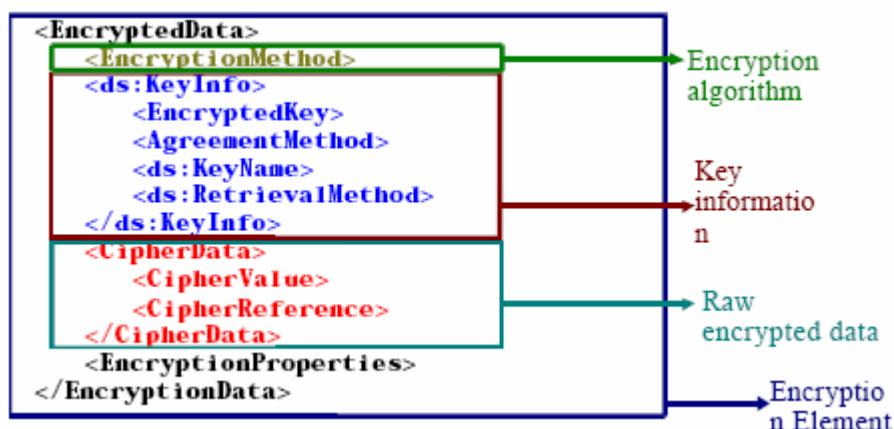
การเข้ารหัสแบบอสมมาตร (Asymmetric Key Cryptography)

การเข้ารหัสแบบนี้จะใช้หลักกุญแจคู่ทำการเข้ารหัสและถอดรหัส ซึ่งกุญแจคู่ที่ว่านี้จะประกอบไปด้วย กุญแจส่วนตัว (private key) และกุญแจสาธารณะ (public key) โดยกุญแจสาธารณะจะใช้ในการเข้ารหัส ส่วนกุญแจส่วนตัวจะใช้ในการถอดรหัส กุญแจสาธารณะนี้สามารถส่งมอบให้กับผู้อื่นได้ เช่น เพื่อนร่วมงานที่เราต้องการติดต่อด้วย หรือแม้กระทั่งวางไว้บนเว็บไซต์เพื่อให้ผู้อื่นสามารถดาวน์โหลดไปใช้งานได้ สำหรับกุญแจส่วนตัวนั้นต้องเก็บไว้กับผู้เป็นเจ้าของกุญแจส่วนตัวเท่านั้น และห้ามเปิดเผยให้ผู้อื่นทราบโดยเด็ดขาด



รูปที่ 4.10 การเข้ารหัสแบบอสมมาตร

4.3.2 อีลีเมนต์ที่ใช้ใน XML Encryption



รูปที่ 4.11 อีลีเมนต์ที่ใช้ใน XML Encryption

อีลีเมนต์ของ XML ในการอธิบายมาตรฐานนี้คือ <EncryptedData>, <EncryptedKey> ในส่วนของบอดีของเมสเสจจะประกอบด้วย <EncryptedData> กับอีลีเมนต์ย่อยอีก 3 อีลีเมนต์ คือ

1. อีลีเมนต์แรกคือ <EncryptionMethod> จะบอกถึงอัลกอริทึมในการเข้ารหัสข้อมูล ในกรณีนี้ อัลกอริทึมชนิด TripleDES จะถูกเลือก และกุญแจที่เหมือนกันจะถูกใช้ในการเข้ารหัสและถอดรหัส
2. อีลีเมนต์ที่สองคือ <KeyInfo> มันจะสมมติว่าทั้งสองฝั่งที่ติดต่อกันนั้นรู้กุญแจที่ใช้ในการเข้ารหัส และถอดรหัส ตัวอย่างชื่อของกุญแจ เช่น "CN=Key13, C=US" ไม่ว่าชื่อใดก็ตามที่ถูกใช้นั้น สิ่งที่สำคัญที่สุดคือทั้งสองฝั่งต้องเข้าใจสิ่งที่กุญแจนี้อ้างถึง
3. อีลีเมนต์สุดท้ายคือ <CipherData> เป็นอีลีเมนต์ย่อยที่จะมี <CipherValue> เป็นอีลีเมนต์ย่อยอีกทีที่จะเป็นการเข้ารหัสข้อมูลที่แท้จริง

ตัวอย่างในกรณีนี้เมื่อข้อมูลถูกเข้ารหัสโดยใช้กุญแจสมมาตร โดยใช้ TripleDES ดังรูปด้านล่าง

```

<s:Envelope
  xmlns:s="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
  xmlns:xenc="http://www.w3.org/2001/04/xmlenc#">
  <s:Body>
    <xenc:EncryptedData>
      <xenc:EncryptionMethod
        Algorithm='http://www.w3.org/2001/04/xmlenc#tripledes-cbc'/>
      <ds:KeyInfo>
        <ds:KeyName> CN=Key13, C=US </ds:KeyName>
      </ds:KeyInfo>
      <xenc:CipherData>
        <xenc:CipherValue> r5KipsDV . . . </xenc:CipherValue>
      </xenc:CipherData>
    </xenc:EncryptedData>
  </s:Body>
</s:Envelope>

```

รูปที่ 4.12 การเข้ารหัสในส่วนบอดีของ SOAP MESSAGE

เมื่อกุญแจสมมาตรถูกเข้ารหัสโดยใช้กุญแจสาธารณะแล้วทำการส่งข้อมูลต่อไป เมื่อได้รับเมสเสจแล้ว ผู้รับสามารถใช้กุญแจส่วนตัวในการถอดรหัส เพื่อดึงเอากุญแจสมมาตรออกมา แล้วใช้กุญแจสมมาตรนี้ในการถอดรหัสข้อมูลที่แท้จริง การส่งกุญแจที่เข้ารหัสนั้น XML Encryption จะกำหนดอิลีเมนต์ <EncryptedKey> เพื่อใช้กับ SOAP เมสเสจ ซึ่ง WS-Security จะกำหนดว่าอิลีเมนต์นี้จะอยู่ใน <Security> เฮดเดอร์ ดังนั้น SOAP เมสเสจจะมีทั้งข้อมูลที่เข้ารหัสแล้ว และกุญแจสมมาตรที่เข้ารหัสแล้ว แสดงดังรูปด้านล่าง

```

<s:Envelope
  xmlns:s="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:wsse="http://schemas.xmlsoap.org/ws/2002/12/secext"
  xmlns:wsu="http://schemas.xmlsoap.org/ws/2002/07/utility"
  xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
  xmlns:xenc="http://www.w3.org/2001/04/xmlenc#">
  <s:Header>
    <wsse:Security>
      <xenc:EncryptedKey>
        <xenc:EncryptionMethod
          Algorithm="http://www.w3.org/2001/04/xmlenc#rsa-1_5"/>
        <ds:KeyInfo>
          <ds:KeyName> CN=Key13, C=US </ds:KeyName>
        </ds:KeyInfo>
        <xenc:CipherData>
          <xenc:CipherValue> ir4DfG . . . </xenc:CipherValue>
        </xenc:CipherData>
        <xenc:ReferenceList>
          <xenc:DataReference URI="#EncryptedBody"/>
        </xenc:ReferenceList>
      </xenc:EncryptedKey>
    </wsse:Security>
  </s:Header>
  <s:Body>
    <xenc:EncryptedData wsu:Id= "EncryptedBody">
      <xenc:EncryptionMethod
        Algorithm="http://www.w3.org/2001/04/xmlenc#tripleDES-cbc"/>
      <xenc:CipherData>
        <xenc:CipherValue> r5KipsDV . . . </xenc:CipherValue>
      </xenc:CipherData>
    </xenc:EncryptedData>
  </s:Body>
</s:Envelope>

```

รูปที่ 4.13 การเข้ารหัส SOAP Message ด้วยกุญแจ

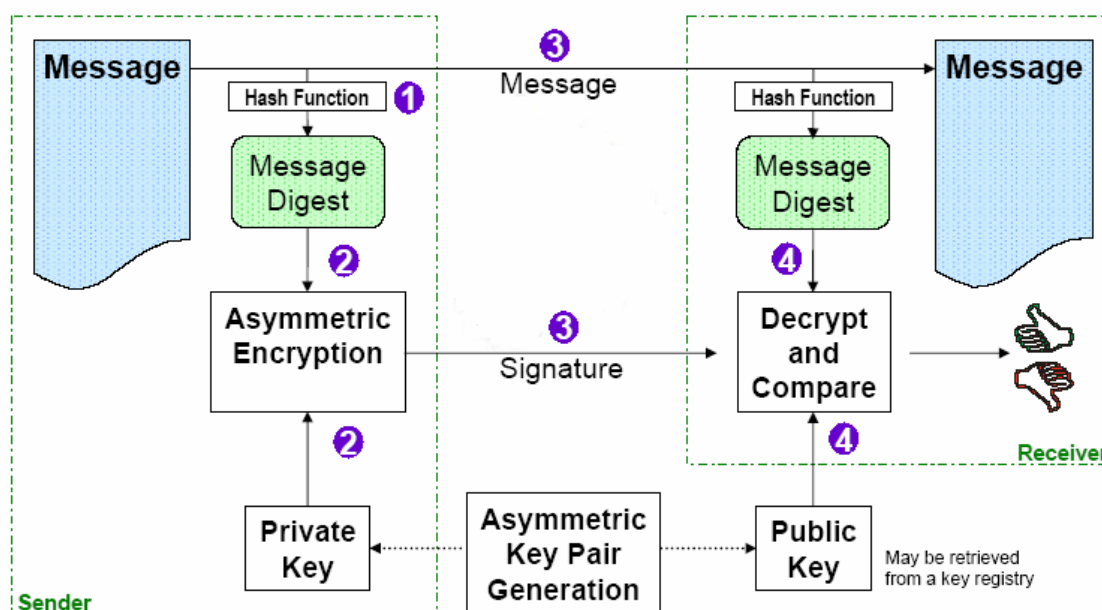
4.4 การรักษาความสมบูรณ์ของข้อมูล (Integrity)

การรักษาความสมบูรณ์ของข้อมูล คือ การรับรองว่าข้อมูลจะไม่ถูกเปลี่ยนแปลงหรือทำลายไปจากต้นฉบับ (Source) ไม่ว่าจะโดยบังเอิญหรือคัดแปลงโดยเจตนาที่อาจส่งผลเสียต่อข้อมูล การคุกคามความสมบูรณ์ของข้อมูล คือ การที่บุคคลที่ไม่ได้รับอนุญาตสามารถที่จะเข้าควบคุมการจัดการของข้อมูลได้ ใน WS-Security การรักษาความสมบูรณ์ของ SOAP Message จะทำผ่าน XML Digital Signature ซึ่งเป็นมาตรฐานของ W3C

4.4.1 หลักการทำงานของ XML Digital Signature

XML Digital Signature เป็นการระบุลายเซ็นดิจิทัลลงบน SOAP Message เพื่อยืนยันตัวตนของผู้ส่งเพื่อพิสูจน์ว่าเอกสารที่ส่งมาไม่ถูกเปลี่ยนแปลงซึ่งถ้าหากเอกสารถูกเปลี่ยนแปลงระหว่างการส่งเมื่อทำการตรวจสอบจะได้ผลไม่ตรงกัน ทำให้บอกได้ว่าเอกสารนี้ถูกส่งจากผู้ส่งที่เราคาดหวังจริงหรือไม่

Digital Signature เป็นการนำหลักการของการทำงานของระบบการเข้ารหัสแบบใช้คู่รหัสกุญแจเพื่อการพิสูจน์ตัวตนมาประยุกต์ใช้ ระบบของลายเซ็นดิจิทัลสามารถแบ่งเป็นขั้นตอนได้ดังนี้

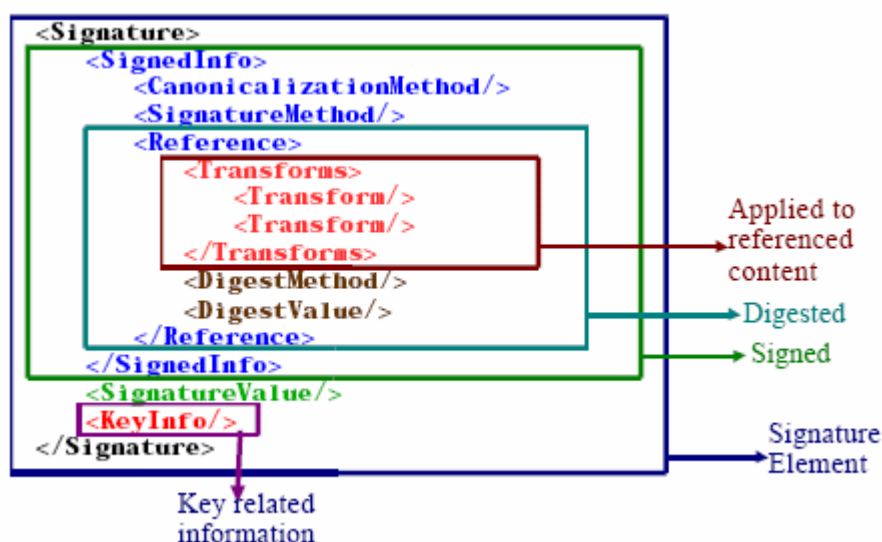


รูปที่ 4.14 ขั้นตอนการทำ Digital Signature

1. เมื่อผู้ใช้ต้องการจะส่งข้อมูลไปยังผู้รับ ข้อมูลนั้นจะถูกนำไปเข้าฟังก์ชันทางคณิตศาสตร์ที่เรียกว่า "แฮชฟังก์ชัน" ได้เป็นเมสเสจไดเจสต์ (Message Digest) ออกมา

2. นำเมสเสจไคเจสต์ที่ได้มาทำการเข้ารหัสด้วยกุญแจส่วนตัว ผลลัพธ์ที่ได้ออกมาจะอยู่ในรูป Signature
3. ส่ง Signature และ message ไปยังฝั่งผู้รับ
4. ในฝั่งผู้รับจะมีการตรวจสอบความถูกต้องของข้อมูลและตรวจสอบว่าข้อมูลถูกส่งมาจากผู้ส่งคนนั้นจริงหรือไม่ โดยการนำ message มาผ่านแฮชฟังก์ชันเพื่อคำนวณค่าเมสเสจไคเจสต์ และทำการถอดรหัสลายเซ็นอิเล็กทรอนิกส์ด้วยกุญแจสาธารณะของผู้ส่ง ถ้าสามารถถอดรหัสได้อย่างถูกต้อง จะเป็นการยืนยันข้อมูลว่ามาจากผู้ส่งคนนั้นจริง และถ้าข้อมูลเมสเสจไคเจสต์ที่ได้จากการถอดรหัสเท่ากับค่าเมสเสจไคเจสต์ที่ได้จากการเข้าแฮชฟังก์ชัน จะถือว่าข้อมูลดังกล่าวนั้นถูกต้อง

4.4.2 อีลีเมนต์ที่ใช้ใน XML Digital Signature



รูปที่ 4.15 อีลีเมนต์ที่ใช้ใน XML Digital Signature

อีลีเมนต์ที่สำคัญในการทำลายเซ็นอิเล็กทรอนิกส์คือ <Signature> เป็นอีลีเมนต์เริ่มต้นที่จะประกอบด้วยอีลีเมนต์ย่อย ที่จะใช้ในการทำลายเซ็นอิเล็กทรอนิกส์ โดยมีอีลีเมนต์ภายในเช่น

- <SignedInfo> ประกอบด้วยรายละเอียดของอินฟอร์เมชันที่เกี่ยวข้องกับการ signed
- <SignatureValue> เป็นตัวระบุถึงของ signature ที่ได้
- <KeyInfo> เป็นตัวที่จะบอกว่าควรจะใช้กุญแจดอกไหนในการในการทำลายเซ็นอิเล็กทรอนิกส์ภายในจะมีอีลีเมนต์ <SecurityTokenReference> ที่ใช้ในการอ้างอิงถึง security token ที่ใช้

```

<?xml version="1.0" encoding="utf-8"?>
<S11:Envelope xmlns:S11="..." xmlns:wsse="..." xmlns:wsu="..."
xmlns:ds="...">
  <S11:Header>
    <wsse:Security>
      <wsse:BinarySecurityToken
        ValueType="...#X509v3"
        EncodingType="...#Base64Binary"
        wsu:Id="X509Token">
        MIIIEZzCCA9CgAwIBAgIQEmtJZc0rqrKh5i...
      </wsse:BinarySecurityToken>
      <ds:Signature>
        <ds:SignedInfo>
          <ds:CanonicalizationMethod Algorithm=
            "http://www.w3.org/2001/10/xml-exc-c14n#" />
          <ds:SignatureMethod Algorithm=
            "http://www.w3.org/2000/09/xmldsig#rsa-sha1" />
          <ds:Reference URI="#myBody">
            <ds:Transforms>
              <ds:Transform Algorithm=
                "http://www.w3.org/2001/10/xml-exc-c14n#" />
            </ds:Transforms>
            <ds:DigestMethod Algorithm=
              "http://www.w3.org/2000/09/xmldsig#sha1" />
            <ds:DigestValue>EULddytSol...</ds:DigestValue>
          </ds:Reference>
        </ds:SignedInfo>
        <ds:SignatureValue>
          BL8jdfToEb1l/vXcMZNNjPOV...
        </ds:SignatureValue>
        <ds:KeyInfo>
          <wsse:SecurityTokenReference>
            <wsse:Reference URI="#X509Token" />
          </wsse:SecurityTokenReference>
        </ds:KeyInfo>
      </ds:Signature>
    </wsse:Security>
  </S11:Header>
  <S11:Body wsu:Id="myBody">
    <tru:StockSymbol xmlns:tru="http://www.fabrikam123.com/payloads">
      QQQ
    </tru:StockSymbol>
  </S11:Body>
</S11:Envelope>

```

รูปที่ 4.16 การสร้างลายเซ็นอิเล็กทรอนิกส์

4.5 ใบรับรองอิเล็กทรอนิกส์ (Certification)

ในโลกของอิเล็กทรอนิกส์ เราสามารถทำการตรวจสอบสถานะของบุคคลที่ประสงค์จะติดต่อได้จากใบรับรองอิเล็กทรอนิกส์ (Certificate) ซึ่งใช้สำหรับการยืนยันตัวตนบุคคล ใบรับรองฯ ดังกล่าวเป็นข้อมูลในรูปแบบอิเล็กทรอนิกส์ที่ออกให้โดยผู้ให้บริการออกใบรับรอง (Certification Authority - CA) ซึ่งต้องเป็นหน่วยงานหรือองค์กรที่เชื่อถือได้และมีความรู้ความเชี่ยวชาญในเทคโนโลยีความปลอดภัย โดยอาศัยเทคโนโลยีหลัก เรียกว่า เทคโนโลยีโครงสร้างพื้นฐานระบบกุญแจสาธารณะ (Public Key Infrastructure - PKI) ซึ่งมีระบบกุญแจคู่ อันประกอบด้วยกุญแจ

ส่วนตัว (Private Key) และกุญแจสาธารณะ (Public Key) เป็นพื้นฐานสำคัญ โดยที่ผู้ให้บริการออกใบรับรองจะทำการรับรองข้อมูลต่างๆ รวมทั้งกุญแจสาธารณะของบุคคลนั้น ซึ่งกุญแจสาธารณะดังกล่าว จะใช้ในการเข้ารหัสข้อมูล (Encryption) และตรวจสอบลายมือชื่อดิจิทัล (Digital Signature) เพื่อเป็นการรักษาความลับ ตรวจสอบความถูกต้อง และที่มาของข้อมูลว่าบุคคลใดเป็นผู้ทำการส่งข้อมูลหรือลงลายมือชื่อดิจิทัล โดยที่ผู้ใช้งานควรที่จะมีความรู้ความเข้าใจเกี่ยวกับกระบวนการในการใช้งานใบรับรองอิเล็กทรอนิกส์ และข้อมูลที่ปรากฏอยู่ในใบรับรองอิเล็กทรอนิกส์ เพื่อก่อให้เกิดประสิทธิภาพในการใช้งาน รวมทั้งยังเป็นประโยชน์แก่ผู้ใช้งานอีกด้วย

รูปแบบของใบรับรองอิเล็กทรอนิกส์ อิงตามมาตรฐาน X.509 Certificate ซึ่งเป็นมาตรฐานของใบรับรองอิเล็กทรอนิกส์ที่กำหนดโดย ITU-T X.509 International Standard ที่บอกถึงรูปแบบของใบรับรองฯ โดยประกอบไปด้วยข้อมูลหลักๆ 2 ส่วน คือ ข้อมูลพื้นฐานของใบรับรองอิเล็กทรอนิกส์ (Basic Certificate Fields) และข้อมูลเพิ่มเติมของใบรับรองอิเล็กทรอนิกส์ (Certificate Extension) ดังรูป

Certificate format version		
Certificate serial number		
Signature algorithm identifier for CA		
Issuer X.500 name		
Validity period		
Subject X.500 Name		
Subject public key information		
Type	Criticality	Value
Type	Criticality	Value 
CA Signature		

รูปที่ 4.17 รูปแบบของใบรับรองอิเล็กทรอนิกส์

ใน การขอเอกสารสิทธิ์นั้นจะต้องกระทำการบนกระบวนการที่ปลอดภัย ซึ่งปัจจุบันมี 2 วิธี คือ

1. การขอเอกสารสิทธิ์ผ่านโปรแกรมประเภทเบราว์เซอร์ เช่น อินเทอร์เน็ตเอ็กซ์พลอเรอร์ เน็ตสเคป เป็นต้น โดยผู้ใช้เข้าไปยังโฮสต์ที่ให้บริการการขอเอกสารสิทธิ์หรือเว็บที่เปิด

ให้บริการดังกล่าวอยู่ ตัวโปรแกรมประเภทบราวเซอร์จะทำการสร้างกุญแจคู่ กุญแจส่วนตัวจะเก็บไว้ที่เครื่องของเรา จากนั้นจะส่งกุญแจสาธารณะและข้อมูลส่วนตัวของเราไปยังโฮสต์หรือเซิร์ฟเวอร์ที่เป็นขององค์กรพิสูจน์ จากนั้นองค์กรพิสูจน์จะทำการรับรองข้อมูลแล้วจึงส่งเอกสารสิทธิ์กลับมาที่เครื่องเรา

2. การขอเอกสารสิทธิ์อีกรูปแบบหนึ่งคือการส่งเอกสารสิทธิ์ที่ไม่ผ่านระบบเครือข่าย กล่าวคือผู้ขอต้องไปทำการขอที่สำนักงานขององค์กรพิสูจน์สิทธิ์โดยตรง โดยทำเรื่องขอเอกสารสิทธิ์และกรอกข้อมูลส่วนตัว องค์กรพิสูจน์สิทธิ์จะทำการรับรองและบันทึกข้อมูลของผู้ขอแล้วทำการส่งเอกสารสิทธิ์พร้อมทั้งกุญแจส่วนตัวให้กับผู้ขอในรูปของไฟล์แผ่นดิสก์ให้ผู้ขอนำไปบันทึกในเครื่องของตนต่อไป

4.5.1 ขั้นตอนการขอใบรับรองของเว็บ <http://www.verisign.com> (CA International)

1. สร้าง CSR(Certificate Signing Request) ขึ้นมา
2. ขั้นตอนการพิจารณาข้อมูลของผู้ที่จะทำการขอ Certificate ซึ่งใช้เวลาตั้งแต่ 1-30 วัน ในการ พิจารณาขึ้นอยู่กับชนิดของ Certificate และข้อมูล เอกสารที่ผู้ร้องขอส่งไปให้ทาง CA
3. จะได้รับ Certificate เป็น Text file จากทาง email

4.5.2 ขั้นตอนการขอใบรับรองของเว็บ <http://ca.khonThai.com> (CA Thai)

1. ขอรหัส PIN ที่ website www.dopa.go.th หรือ www.khonThai.com ซึ่งเป็น website ของกรมการปกครอง
 ** ในกรณีที่ได้ลงทะเบียนขอรหัส PIN แล้วมีเพียงรหัส PIN1 ให้ใส่รหัส PIN2 เป็นรหัสเดียวกับ PIN1 แล้วสามารถเข้าไปเปลี่ยนรหัส PIN2 ได้ใน website ของกรมการปกครอง ***
 2. ลงทะเบียนที่หน้าลงทะเบียนของ website ของ KhonThai CA
 3. ใส่ PID (หมายเลขบัตรประชาชน 13 หลัก) และใส่รหัส PIN1, PIN2 ให้ถูกต้อง
 4. ถ้า PID และรหัส PIN ถูกต้องจะปรากฏข้อมูลของคุณ
 5. ยืนยันการลงทะเบียนโดย
 - ใส่ E-mail address ที่ต้องการใช้กับใบรับรองนี้ กล่าวคือใบรับรอง 1 ใบจะใช้ได้กับ 1 E-mail address เท่านั้น
 - ใส่องค์กรที่คุณสังกัดอยู่ ในกรณีไม่ได้สังกัดองค์กรใดไม่ต้องใส่ข้อมูล
 - ตั้งรหัสผ่านและยืนยันรหัสผ่าน รหัสผ่านนี้จะต้องนำไปใช้เมื่อต้องการนำกุญแจส่วนตัวมาใช้หรือขอใช้ บริการอื่นๆเพื่อยืนยันว่าคุณเป็นเจ้าใบรับรองที่แท้จริง
- เมื่อทำการลงทะเบียนเรียบร้อยแล้วทาง KhonThai CA จะอนุญาตให้ผู้ขอใบรับรองใช้ใบรับรองดังกล่าวได้ทันที

สิ่งที่ได้รับหลังจากลงทะเบียน

- สิทธิในการใช้ใบรับรอง โดยใบรับรองจะมีกุญแจสาธารณะของคุณอยู่
- สิทธิในการใช้กุญแจส่วนตัว (Private Key)

ทั้งใบรับรองและกุญแจส่วนตัวนั้นจะยังอยู่บน Server ของ Khonthai CA เพื่อความปลอดภัยจนกระทั่งเจ้าของใบรับรองต้องการใช้งาน Khonthai CA จะมีวิธีการนำออกมาใช้งานแบบเฉพาะของ Khonthai CA ที่มีความปลอดภัยสูงซึ่งจะกล่าวถึงต่อไป

สิ่งที่ควรปฏิบัติหลังจากได้ใบรับรองแล้ว

- ตรวจสอบรายละเอียดและสถานะของใบรับรองโดยการ "ค้นหาใบรับรอง" จากหน้าค้นหา

4.6 Secure Sockets Layer (SSL)

HTTPS จะใช้การพิสูจน์ตัวตนระหว่างฝั่ง client และฝั่ง server ผ่านเอกสารรับรอง certification ไม่ว่าจะเป็นแบบตั้งขึ้นเอง (self-signed) หรือแบบที่ขอจากผู้ที่มีความน่าเชื่อถือ (certification agency) สำหรับเว็บเซิร์ฟเวอร์ที่ทำงานอยู่บนโปรโตคอล HTTP นั้น HTTPS/SSL สามารถที่จะถูกนำมาใช้ร่วมกับการรักษาความปลอดภัยในระดับ message (message-level) ซึ่ง SSL ถือว่าเป็นความปลอดภัยในระดับ transport (transport-level) โดย HTTPS จะทำการเข้ารหัส HTTP data packet ทั้งหมด

SSL เป็นโปรโตคอลระดับ Application นิยมใช้ในการเพิ่มความปลอดภัยของการสื่อสารผ่านโปรโตคอล HTTP ซึ่งเป็นการสื่อสารผ่านเว็บ ทำให้เกิดการสื่อสารอย่างปลอดภัยระหว่าง Client และ Server โดยหลักการทำงานแบ่งออกเป็น 3 ส่วนใหญ่ๆ คือ

1. การตรวจสอบ server ว่าเป็นตัวจริง : ตัวโปรแกรม client ที่มีขีดความสามารถในการสื่อสารแบบ SSL จะสามารถตรวจสอบเครื่อง server ที่ตนกำลังจะไปเชื่อมต่อได้ว่า server นั้นเป็น server ตัวจริงหรือไม่ โดยใช้เทคนิคการเข้ารหัสแบบ public key ในการตรวจสอบใบรับรอง (certificate) และ public ID ของ server นั้น (โดยที่มีองค์กรที่ client เชื่อถือเป็นผู้ออกใบรับรองและ public ID ให้แก่ server นั้น)

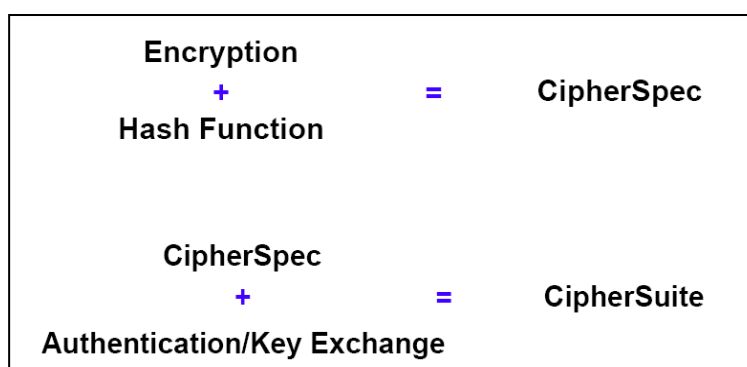
หน้าที่นี้ของ SSL เป็นหน้าที่ที่สำคัญ โดยเฉพาะอย่างยิ่งในกรณีที่ client ต้องการที่จะส่งข้อมูลที่เป็นความลับ (เช่น หมายเลข credit card) ให้กับ server ซึ่ง client จะต้องตรวจสอบก่อนว่า server เป็นตัวจริงหรือไม่

2. การตรวจสอบว่า client เป็นตัวจริง: server ที่มีขีดความสามารถในการสื่อสารแบบ SSL จะใช้เทคนิคเช่นเดียวกับในหัวข้อที่แล้วในการตรวจสอบ client หรือผู้เข้าว่าเป็นตัวจริงหรือไม่ โดยจะตรวจสอบใบรับรองและ public ID (ที่มีองค์กรที่ server เชื่อถือเป็นผู้ออกให้) ของ client หรือผู้ใช้นั้น

หน้าที่นี้ของ SSL จะมีประโยชน์ในกรณีเช่น ธนาคารต้องการที่จะส่งข้อมูลลับทางการเงินให้แก่ลูกค้าของตนผ่านทางเครือข่าย Internet (server ก็จะต้องตรวจสอบ client ก่อนว่าเป็น client นั้นจริง)

3. การเข้ารหัสลับการเชื่อมต่อ: ในกรณีนี้ ข้อมูลทั้งหมดที่ถูกส่งระหว่าง client และ server จะถูกเข้ารหัสลับ โดยโปรแกรมที่ส่งข้อมูลเป็นผู้เข้ารหัสและโปรแกรมที่รับข้อมูลเป็นผู้ถอดรหัส (โดยใช้วิธี public key) นอกจากการเข้ารหัสลับในลักษณะนี้แล้ว SSL ยังสามารถปกป้องความถูกต้องสมบูรณ์ของข้อมูลได้อีกด้วย กล่าวคือ ตัวโปรแกรมรับข้อมูลจะทราบได้หากข้อมูลถูกเปลี่ยนแปลงไปในขณะกำลังเดินทางจากผู้ส่งไปยังผู้รับ

เมื่อมีการใช้ SSL ผู้ใช้สามารถระบุอัลกอริทึมที่ต้องการในการเข้ารหัส, กำหนด hash function ที่ใช้ในการสร้าง message digest หรือ MAC (message authentication code) ซึ่งทั้งสองสิ่งจะถูกเรียกว่า CipherSpec นอกจากนี้แล้วยังต้องรู้อัลกอริทึมที่ใช้สำหรับการพิสูจน์ตัวตน (Authentication) และ key ที่แลกเปลี่ยนกัน โดย authentication/key exchange และ cipherspec จะถูกเรียกว่า CipherSuite แสดงดังภาพ

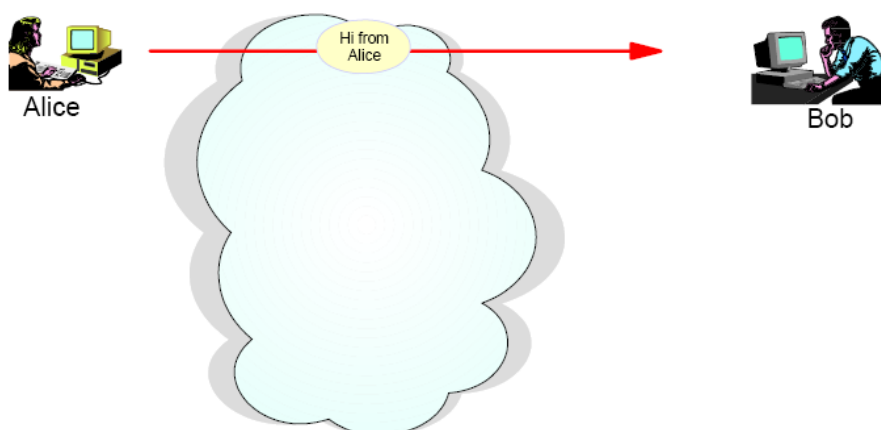


รูปที่ 4.18 ความหมายของ CipherSpec และ CipherSuite

โดยกระบวนการในการเริ่มต้นการสื่อสารผ่านชั้น SSL มี ขั้นตอนดังนี้

1. ประกาศชุดวิธีการเข้ารหัสไคเจสต์และลายเซ็นดิจิทัลที่สนับสนุนของทั้งไคลเอ็นต์และเซิร์ฟเวอร์

ไคลเอ็นต์และเซิร์ฟเวอร์ส่งข้อความเริ่มต้นการสื่อสาร (Hello message) ซึ่งประกอบไปด้วยเวอร์ชันของโพรโทคอลที่ใช้ วิธีการเข้ารหัสที่เว็บเซิร์ฟเวอร์และไคลเอ็นต์สนับสนุน หมายเลขระบุการสื่อสาร (Session identifier) รวมถึงวิธีการบีบอัดข้อมูลในการสื่อสารที่สนับสนุนหมายเลขระบุการสื่อสารที่เกิดขึ้น ใช้สำหรับตรวจสอบการเชื่อมต่อระหว่างไคลเอ็นต์และเซิร์ฟเวอร์ ถ้ามีการเชื่อมต่อก่อนหน้านี้เกิดขึ้น แสดงว่าได้มีการตกลงวิธีการสื่อสารแล้ว สามารถเริ่มต้นส่งข้อมูลได้ทันที เป็นการลดเวลาติดต่อสื่อสารลง



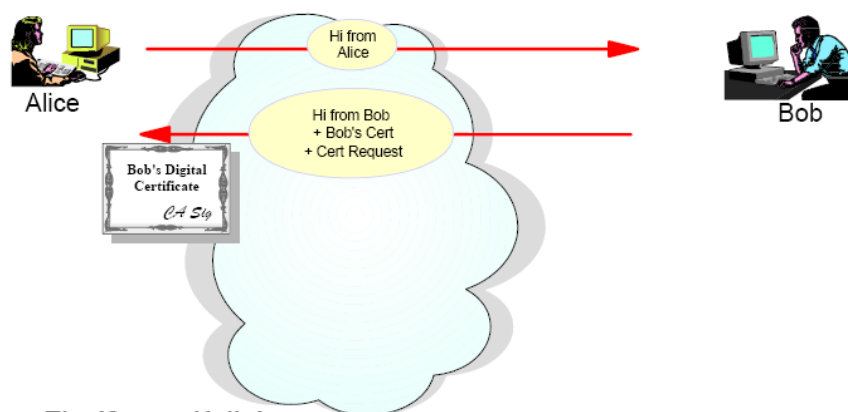
■ The 'Client Hello'

- ▶ Alice sends Bob some random text
- ▶ Also sends what CipherSpecs and compression methods she can use
- ▶ Alice is considered the client since she started the handshake

รูปที่ 4.19 การเริ่มต้นสื่อสารด้วย Hello Message

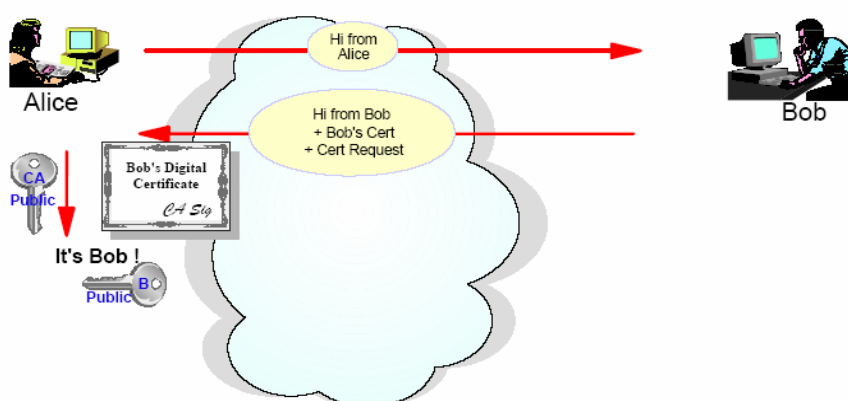
2. การพิสูจน์ตัวตนของเซิร์ฟเวอร์ต่อไคลเอ็นต์

ถัดมาเว็บเซิร์ฟเวอร์ทำการส่ง Certificate หรือใบยืนยันความมีตัวตนของเซิร์ฟเวอร์ ไคลเอ็นต์จะทำการตรวจสอบ Certificate กับผู้ให้บริการ Certificate Authority ที่ได้ตั้งค่าไว้ เพื่อยืนยันความถูกต้องของ Certificate ของเซิร์ฟเวอร์



- The 'Server Hello'
 - ▶ Bob sends Alice some random text
 - ▶ Bob chooses the CipherSpec to be used, from Alice's list
- The Server Certificate
- The Client Certificate Request

รูปที่ 4.20 ขั้นตอน Server Hello

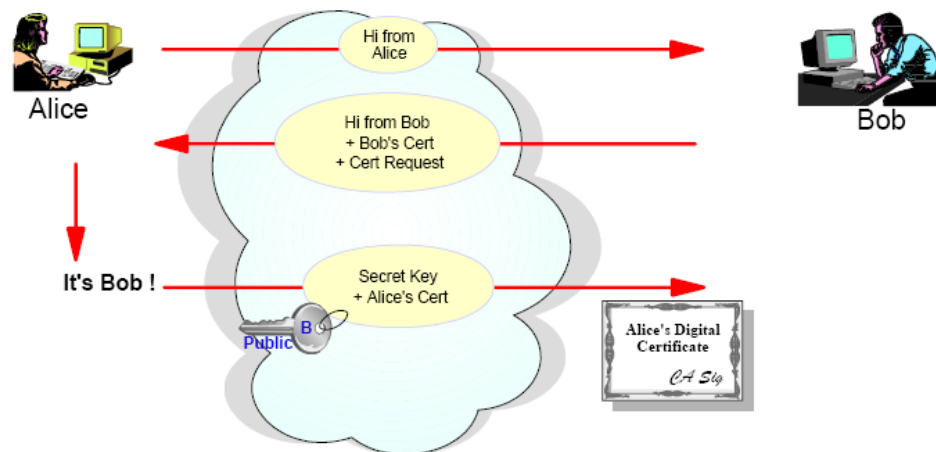


- Verify Server Certificate
 - ▶ Check Validity Period
 - ▶ Decrypt using CA's Public Key - verifies that CA is trusted
 - ▶ Check Domain Name and/or Distinguished Name
 - ▶ Also receives Bob's Public Key

รูปที่ 4.21 ขั้นตอนการตรวจสอบ certificate ของ server

3. การพิสูจน์ตัวตนของไคลเอนต์ต่อเซิร์ฟเวอร์ ถ้าจำเป็น

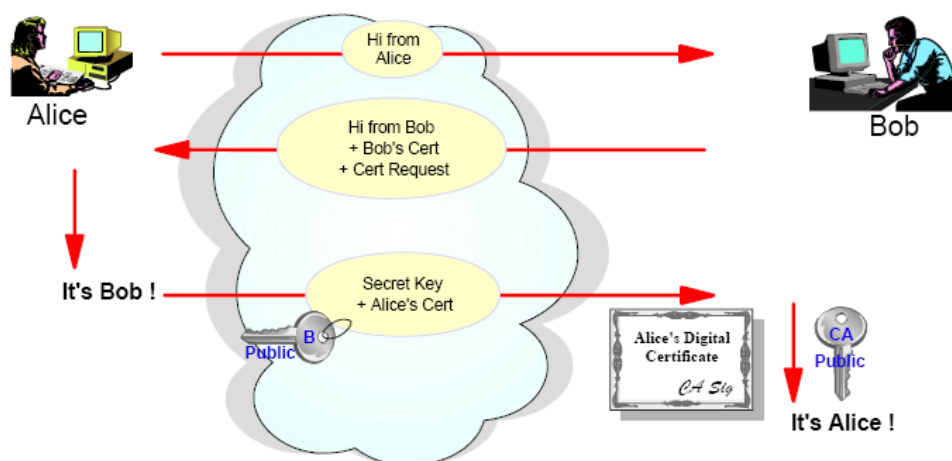
เซิร์ฟเวอร์สามารถร้องขอ Certificate จากไคลเอนต์เพื่อตรวจสอบความถูกต้องของ Client ด้วยก็ได้ โดยใช้ในกรณีที่มีการจำกัดการใช้งานเฉพาะไคลเอนต์ที่ต้องการเท่านั้น ซึ่ง SSL สนับสนุนการตรวจสอบที่ได้จากทั้งเซิร์ฟเวอร์และไคลเอนต์ ขึ้นอยู่กับการเลือกใช้งานในขณะติดต่อสื่อสารที่เกิดขึ้นนั้น



■ Client Key Exchange

- ▶ Alice sends Bob the Secret Key to use
- ▶ This is encrypted with Bob's Public Key
- ▶ Also sends her certificate

รูปที่ 4.22 ขั้นตอน Client Key Exchange

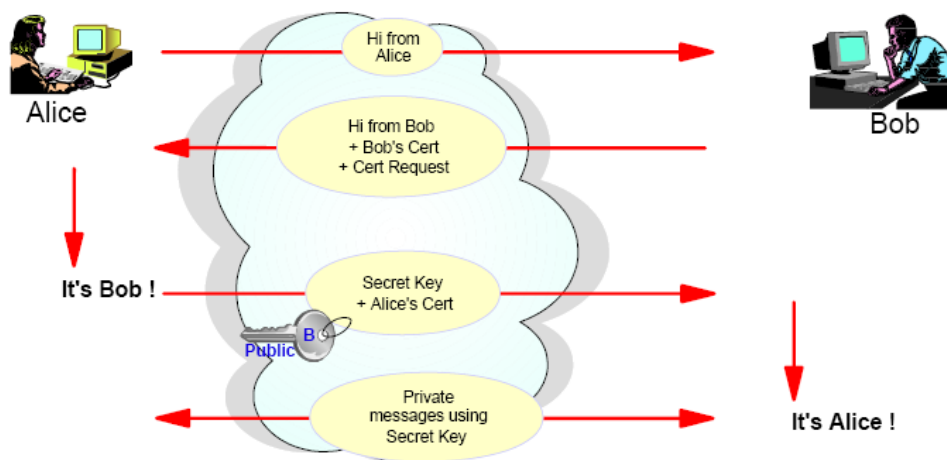


■ Verify Client Certificate

- ▶ Decrypt using CA's public Key

รูปที่ 4.23 ขั้นตอนตรวจสอบ Client Certification

4. โคลเอ็นต์และเซิร์ฟเวอร์ตกลงชุดวิธีการเข้ารหัสการสร้างไคเจสต์ และการใช้ลายเซ็นดิจิทัล



- Send Information using agreed Secret Key
 - ▶ Randomly generated 1-time key
- This is now a 'secure line'

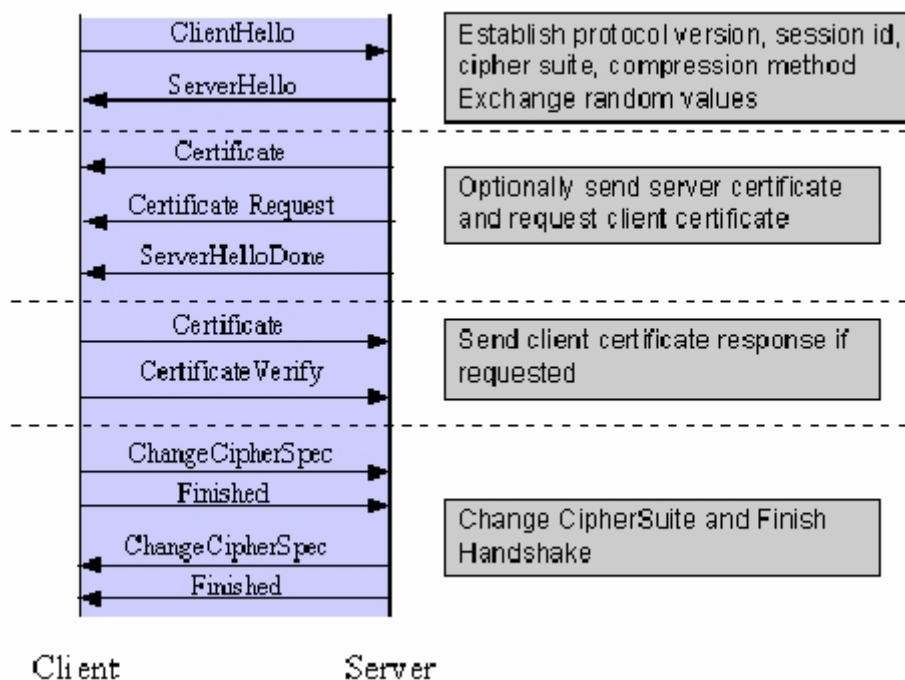
รูปที่ 4.24 การขั้นตอนสุดท้ายของ SSL

วิธีการแลกเปลี่ยนกุญแจในการเข้ารหัส (Key exchange method) คือการกำหนดกลไกการแลกเปลี่ยนกุญแจที่ใช้ในการเข้ารหัสระหว่างการสื่อสาร โดยทั้งโคลเอ็นต์และเซิร์ฟเวอร์จะใช้กุญแจนี้ในการเข้ารหัสและถอดรหัสข้อมูล ใน SSL เวอร์ชัน 2.0 จะสนับสนุนวิธีการแลกเปลี่ยนกุญแจแบบ RSA ส่วน SSL เวอร์ชัน 3.0 ขึ้นไปจะสนับสนุนวิธีการอื่นๆ เพิ่มเติมเช่นการใช้ RSA ร่วมกับการใช้ Certificate หรือ Diffie-Hellman เป็นต้น

วิธีการเข้ารหัสในปัจจุบันแบ่งเป็นสองวิธีคือ การใช้กุญแจเดียวกันในการเข้ารหัสและถอดรหัส อาจเรียกกุญแจนี้ว่า Session key หรือ Secret key ส่วนอีกวิธีการคือ การใช้กุญแจคนละตัวในการเข้ารหัสและถอดรหัส ประกอบไปด้วยกุญแจสาธารณะและกุญแจส่วนตัวซึ่งเป็นที่คุ้นเคยกันเสมอ การเข้ารหัสด้วยกุญแจใด จะต้องถอดรหัสด้วยกุญแจที่คู่กันและตรงกันข้ามเท่านั้น มักใช้วิธีการเข้ารหัสด้วยกุญแจคนละตัวมาใช้ในการเข้ารหัส Session key และส่งไปให้ฝั่งตรงข้ามก่อนการสื่อสารจะเกิดขึ้นรวมเรียกว่าวิธีการแลกเปลี่ยนกุญแจในการเข้ารหัส

SSL ใช้วิธีการเข้ารหัสด้วยกุญแจสมมาตร หรือกุญแจเดียวในการเข้ารหัสและถอดรหัส ตามที่กล่าวข้างต้น วิธีการเข้ารหัสคือ การเข้ารหัสด้วย DES และ 3DES (Data Encryption Standard), วิธีการเข้ารหัสด้วย IDEA ส่วน RC2 และ RC4 เป็นวิธีการเข้ารหัสของ RSA รวมถึงวิธีการเข้ารหัสแบบ Fortezza สำหรับความยาวของการเข้ารหัสที่ใช้คือ 40 บิต, 96 บิต และ 128 บิต

การสร้าง Message Authentication Code (MAC) เพื่อใช้สำหรับการยืนยันความถูกต้องของข้อมูลระหว่างการสื่อสารและป้องกันการปลอมข้อมูล ส่วนฟังก์ชันสร้างไคเจสต์ที่ SSL สนับสนุนและเลือกใช้ได้ในปัจจุบันคือ MD5 ขนาด 128 บิต และ SHA-1 (Secure Hash Algorithm) ขนาด 160 บิต ซึ่งจะได้วิธีการที่ทั้งสองฝ่ายสนับสนุนและเหมาะสมซึ่งเป็นขั้นตอนสุดท้ายก่อนการสื่อสารที่มีการเข้ารหัสจะเริ่มต้นขึ้น



รูปที่ 4.25 ภาพรวมกระบวนการเริ่มต้นการติดต่อสื่อสารของโปรโตคอล SSL

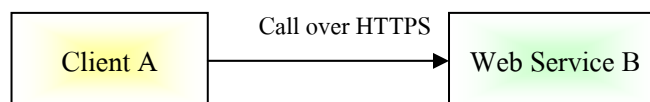
ข้อดีของ SSL

1. มีการเตรียมโปรโตคอลสำหรับฟังก์ชันที่เราต้องการ
 - Encryption
 - Message Integrity Checking
 - Authentication
2. ใช้ Public/Private Key
 - ไม่มีปัญหาเรื่องการกระจาย key
3. ได้รับการยอมรับอย่างกว้างขวางในสังคม Internet

4.7 ความจำเป็นที่จะต้องใช้ WS-Security คู่ไปกับ Secure Sockets Layer (SSL)

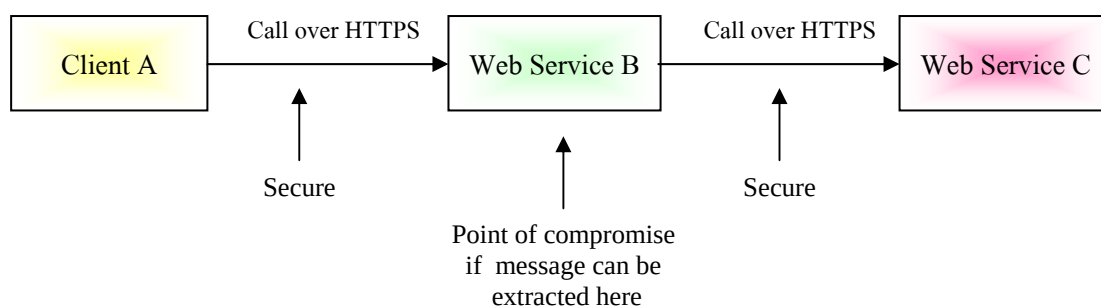
เนื่องจาก SSL นั้นยังมีข้อเสียอยู่บางอย่างที่ไม่เหมาะสมจะนำมาใช้กับเว็บเซอร์วิส โดยมีเหตุผลหลัก 2 ข้อ ดังต่อไปนี้

1. SSL นั้นเป็นการรักษาความปลอดภัยแบบ จุด-จุด (point-to-point security) นั้นหมายถึงว่า จะมีความปลอดภัยแต่จากจุดเริ่มถึงจุดที่มันส่งไปหา ซึ่งการรักษาความปลอดภัยแบบนี้จะ ใช้ได้แค่กรณีแบบติดต่อกันทีละแค่ 2 เครื่องเท่านั้น ดังรูปด้านล่าง



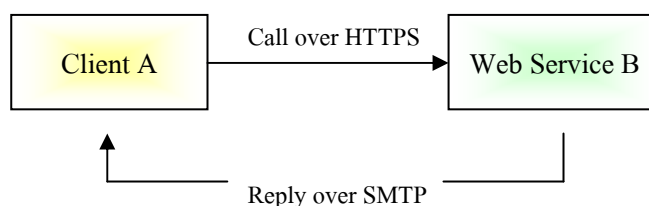
รูปที่ 4.26 การรักษาความปลอดภัยแบบ จุด-จุด บน HTTPS

แต่ถ้าเว็บเซอร์วิสส่งต่อไปให้เว็บเซอร์วิสอื่น กรณีนี้ถึงแม้ว่าจะมีความปลอดภัยจากเอไปบี และจากบีไปซีก็ตามแต่เราก็ไม่สามารถมั่นใจได้เลยว่าข้อมูลที่ถูกส่งจากเอไปซีจะมีความปลอดภัย เพราะข้อมูลสามารถเกิดการเปลี่ยนได้ในเว็บเซอร์วิสบี ดังรูปด้านล่าง



รูปที่ 4.27 การรักษาความปลอดภัยแบบ จุด-จุดบน HTTP และการส่งต่อ

2. เว็บเซอร์วิสนั้นมีความจำเป็นที่ต้องทำงานบนโปรโตคอลหลายอย่าง ซึ่ง SSL นั้นทำงานอยู่บน HTTP เท่านั้น เพราะฉะนั้นเมื่อมีการทำงานบนโปรโตคอลอื่น เช่น SMTP ก็ไม่สามารถที่จะมีความปลอดภัยได้ ดังรูปด้านล่าง



รูปที่ 4.28 การรักษาความปลอดภัยแบบ จุด-จุดบน HTTP และไม่ปลอดภัยบน SMTP

จากข้อเสียข้างต้นนั้น สามารถแก้ไขได้โดยการใช้ควบคู่ไปกับ WS-Security เนื่องจาก WS-Security เป็นวิธีการรักษาความปลอดภัยแบบ จบ-จบ (end-to-end) และยังไม่ยึดติดกับโปรโตคอลใดอีกด้วย